

Practical-4

Aim:

To process the NCDC historical weather dataset using Hadoop MapReduce and determine the maximum temperature recorded for each year, analyze yearly temperature trends, and identify the hottest year in the dataset.

Problem Definition:

A meteorological research organization maintains a large collection of historical weather records collected from weather stations worldwide. Processing such large datasets using traditional sequential methods can be time-consuming and inefficient.

The objective of this practical is to use the Hadoop MapReduce framework to process the NCDC weather dataset. The Mapper extracts the **year** and **temperature** from each weather record and generates intermediate key-value pairs. The Reducer processes all temperature values belonging to the same year and determines the maximum temperature recorded during that year.

The final output is stored in HDFS and used to analyze long-term temperature trends and identify the hottest year in the dataset.

Objectives:

1. To upload the NCDC weather dataset into HDFS.
2. To examine and understand the structure of NCDC weather records.
3. To develop a Mapper program for extracting the year and temperature values.
4. To generate intermediate key-value pairs in the form (Year, Temperature).
5. To develop a Reducer program for finding the maximum temperature for each year.
6. To execute the MapReduce job on Hadoop.
7. To store and verify the generated output in HDFS.
8. To analyze yearly maximum temperature trends.
9. To identify the hottest year in the dataset.

10.To understand the advantages of MapReduce for large-scale climate data analysis.

Observation / Output:

Task 1: Upload the NCDC weather dataset to HDFS and examine the structure of the weather records.

```
gitanshu@Parth:~$ cd ~
gitanshu@Parth:~$ mkdir ncnc_temperature
gitanshu@Parth:~/ncnc_temperature$
gitanshu@Parth:~/ncnc_temperature$ pwd
/home/gitanshu/ncnc_temperature
```

```
gitanshu@Parth:~/ncnc_temperature$ nano weather.txt
gitanshu@Parth:~/ncnc_temperature$ cat weather.txt
2018 25
2018 32
2018 28
2018 35
2019 30
2019 38
2019 34
2019 36
2020 29
2020 40
2020 37
2020 42
2021 35
2021 41
2021 39
2021 44
2022 38
2022 45
2022 42
2022 43
```

```
2022 43
gitanshu@Parth:~/ncnc_temperature$ cat weather.txt | python3 mapper.py | sort | python3 reducer.py
2018 35.0
2019 38.0
2020 42.0
2021 44.0
2022 45.0
gitanshu@Parth:~/ncnc_temperature$ hdfs dfs -mkdir -p /user/gitanshu/ncnc/input
gitanshu@Parth:~/ncnc_temperature$ hdfs dfs -ls /user/gitanshu/ncnc
Found 1 items
drwxr-xr-x - gitanshu supergroup 0 2026-08-14 10:36 /user/gitanshu/ncnc/input
gitanshu@Parth:~/ncnc_temperature$ hdfs dfs -put weather.txt /user/gitanshu/ncnc/input/
gitanshu@Parth:~/ncnc_temperature$ hdfs dfs -ls /user/gitanshu/ncnc/input
Found 1 items
-rw-r--r-- 2 gitanshu supergroup 160 2026-08-14 10:36 /user/gitanshu/ncnc/input/weather.txt
```

Task 2: Develop and implement the Mapper program to extract the year and temperature values from each record and generate intermediate key-value pairs.

```
gitanshu@Parth:~/ncdc_temperature$ nano mapper.py
gitanshu@Parth:~/ncdc_temperature$ chmod +x mapper.py
gitanshu@Parth:~/ncdc_temperature$ ls -l
total 8
-rwxr-xr-x 1 gitanshu gitanshu 226 Aug 14 09:18 mapper.py
-rw-r--r-- 1 gitanshu gitanshu 160 Aug 14 09:16 weather.txt
gitanshu@Parth:~/ncdc_temperature$ nano reducer.py
gitanshu@Parth:~/ncdc_temperature$ chmod +x reducer.py
gitanshu@Parth:~/ncdc_temperature$ cat weather.txt | python3 mapper.py
2018    25
2018    32
2018    28
2018    35
2019    30
2019    38
2019    34
2019    36
2020    29
2020    40
2020    37
2020    42
2021    35
2021    41
2021    39
2021    44
2022    38
2022    45
2022    42
2022    43
```

Task 3: Develop and implement the Reducer program to determine the maximum temperature recorded for each year.

```
gitanshu@Parth:~/ncdc_temperature$ nano reducer.py
gitanshu@Parth:~/ncdc_temperature$ chmod +x reducer.py

2022    43
gitanshu@Parth:~/ncdc_temperature$ cat weather.txt | python3 mapper.py | sort | python3 reducer.py
2018    35.0
2019    38.0
2020    42.0
2021    44.0
2022    45.0
```

Task 4: Execute the MapReduce job on Hadoop and store the results in HDFS. Verify the correctness of the output generated.

```

gitanshu@Parth:~/ncdc_temperature$ hadoop jar $STREAMING_JAR \
  -input /user/gitanshu/ncdc/input \
  -output /user/gitanshu/ncdc/output \
  -mapper "python3 mapper.py" \
  -reducer "python3 reducer.py" \
  -file mapper.py \
  -file reducer.py
2026-08-14 10:44:10,144 WARN streaming.StreamJob: -file option is deprecated, please use generic option -files instead.
2026-08-14 10:44:11,233 INFO client.DefaultHadoopFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2026-08-14 10:44:11,457 INFO client.DefaultHadoopFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2026-08-14 10:44:11,816 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/gitanshu/.staging/job_1786698685755_0002
2026-08-14 10:44:12,489 INFO mapred.FileInputFormat: Total input files to process : 1
2026-08-14 10:44:12,589 INFO mapreduce.JobSubmitter: number of splits:2
2026-08-14 10:44:12,926 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1786698685755_0002
2026-08-14 10:44:12,926 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-08-14 10:44:13,133 INFO conf.Configuration: resource-types.xml not found
2026-08-14 10:44:13,134 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-08-14 10:44:13,676 INFO impl.YarnClientImpl: Submitted application application_1786698685755_0002
2026-08-14 10:44:13,716 INFO mapreduce.Job: The url to track the job: http://Parth.localdomain:8088/proxy/application_1786698685755_0002/
2026-08-14 10:44:13,719 INFO mapreduce.Job: Running job: job_1786698685755_0002
2026-08-14 10:44:23,824 INFO mapreduce.Job: Job job_1786698685755_0002 running in uber mode : false
2026-08-14 10:44:23,825 INFO mapreduce.Job: map 0% reduce 0%
2026-08-14 10:44:34,720 INFO mapreduce.Job: map 100% reduce 0%
2026-08-14 10:44:39,791 INFO mapreduce.Job: map 100% reduce 100%
2026-08-14 10:44:40,810 INFO mapreduce.Job: Job job_1786698685755_0002 completed successfully
2026-08-14 10:44:40,900 INFO mapreduce.Job: Counters: 54

File System Counters
  FILE: Number of bytes read=206
  FILE: Number of bytes written=964761
  FILE: Number of read operations=0
  FILE: Number of large read operations=0
  FILE: Number of write operations=0
  HDFS: Number of bytes read=460
  HDFS: Number of bytes written=50
  HDFS: Number of read operations=11
  HDFS: Number of large read operations=0
  HDFS: Number of write operations=2
  HDFS: Number of bytes read erasure-coded=0

Job Counters
  Launched map tasks=2
  Launched reduce tasks=1
  Data-local map tasks=2
  Total time spent by all maps in occupied slots (ms)=16914
  Total time spent by all reduces in occupied slots (ms)=3374
  Total time spent by all map tasks (ms)=16914
  Total time spent by all reduce tasks (ms)=3374
  Total vcore-milliseconds taken by all map tasks=16914
  Total vcore-milliseconds taken by all reduce tasks=3374
  Total megabyte-milliseconds taken by all map tasks=17319936
  Total megabyte-milliseconds taken by all reduce tasks=3454976

```

```

gitanshu@Parth:~/ncdc_temperature$ hdfs dfs -ls /user/gitanshu/ncdc/output
Found 2 items
-rw-r--r--  2 gitanshu supergroup          0 2026-08-14 10:44 /user/gitanshu/ncdc/output/_SUCCESS
-rw-r--r--  2 gitanshu supergroup        50 2026-08-14 10:44 /user/gitanshu/ncdc/output/part-00000

```

Task 5: Analyze the yearly maximum temperature trends and identify the hottest year in the dataset. Discuss the advantages of using MapReduce for large-scale climate data analysis.

```

gitanshu@Parth:~/ncdc_temperature$ hdfs dfs -cat /user/gitanshu/ncdc/output/part-00000
2018      35.0
2019 <    38.0
2020      42.0
2021      44.0
2022      45.0

```

Key Questions / Analysis:

Q1. How is weather data parsed and processed using MapReduce?

The weather dataset is divided into multiple input records. Each Mapper processes a portion of the input data independently. The Mapper parses each weather record and extracts important fields such as the year and temperature.

The Mapper generates intermediate key-value pairs in the form:

(Year, Temperature)

During the Shuffle and Sort phase, Hadoop groups all values belonging to the same year. The Reducer then receives one year along with all its associated temperature values and calculates the maximum temperature.

The processing flow is:

NCDC Dataset



Input Split



Mapper



(Year, Temperature)



Shuffle and Sort



Year → List of Temperatures



Reducer



(Year, Maximum Temperature)

Q2. What key-value pairs are generated by the Mapper?

The Mapper generates the following type of key-value pair:

Key = Year

Value = Temperature

For example, if a weather record contains the year 2020 and temperature 42°C, the Mapper generates:

2020 42

If multiple weather records belong to the same year, multiple key-value pairs are generated:

2020 38

2020 40

2020 42

2020 39

These values are later grouped by Hadoop before being passed to the Reducer.

Q3. How does the Reducer identify the maximum temperature for a given year?

The Reducer receives a year as the key and all temperature values associated with that year.

For example:

Key: 2020

Values: [38, 40, 42, 39]

The Reducer initializes a variable for the maximum temperature and compares each temperature value.

Conceptually:

Maximum = $-\infty$

For each temperature:

If temperature > Maximum:

Maximum = temperature

After processing all values:

2020 42

Thus, the maximum temperature recorded for the year 2020 is 42°C.

Q4. What challenges arise when processing large climate datasets?

Some major challenges include:

- **Large Data Volume:** Climate datasets may contain records collected over many years from thousands of weather stations.

- **Data Quality:** Records may contain missing, invalid, or corrupted temperature values.
- **Different Data Formats:** Historical datasets may have inconsistent structures or formats.
- **Processing Time:** Sequential processing of huge datasets can take a significant amount of time.
- **Storage Requirements:** Large climate datasets require distributed and scalable storage.
- **Hardware Failures:** In distributed environments, individual machines may fail during processing.
- **Data Transfer Overhead:** Moving large datasets between machines can reduce performance.

Q5. How does Hadoop improve scalability and performance for weather data analytics?

Hadoop improves large-scale weather data processing through:

1. Distributed Storage:

HDFS stores large datasets across multiple machines.

2. Parallel Processing:

Multiple Mapper tasks process different portions of the weather dataset simultaneously.

3. Scalability:

Additional machines can be added to the Hadoop cluster to increase storage and processing capacity.

4. Data Locality:

Hadoop attempts to execute processing tasks close to the location where the data is stored, reducing unnecessary network transfer.

5. Fault Tolerance:

HDFS maintains replicas of data blocks, allowing processing to continue even if a machine fails.

6. Automatic Resource Management:

YARN manages cluster resources and schedules MapReduce tasks efficiently.

Therefore, Hadoop is suitable for processing very large climate datasets that would be difficult or slow to analyze using traditional single-machine processing.

Supplementary Problems:

1. Find the Minimum Temperature Recorded for Each Year

The Mapper remains the same:

(Year, Temperature)

The Reducer is modified to select the minimum temperature instead of the maximum.

Example:

2020 [42, 38, 40, 39]

Output:

2020 38

2. Calculate Average Yearly Temperatures

For calculating the average temperature, the Reducer calculates:

Average Temperature = Sum of Temperatures / Number of Records

Example:

2020 [38, 40, 42, 39]

Sum = 159

Count = 4

Average = 159 / 4

Average = 39.75°C

Output:

2020 39.75

3. Identify the Year with the Highest Recorded Temperature Overall

After obtaining the maximum temperature for every year:

2018 35

2019 38

2020 42

2021 44

2022 45

The results are compared to determine the overall highest temperature.

Therefore:

Hottest Year: 2022

Maximum Temperature: 45°C

Tools / Technology Used:

- **Hadoop:** Hadoop 3.x
- **Distributed Storage:** HDFS
- **Processing Framework:** MapReduce
- **Programming Language:** Java
- **Java Version:** JDK 8 / JDK 11
- **Operating System:** Linux / Ubuntu
- **Development Environment:** Eclipse / IntelliJ IDEA / Terminal
- **Resource Management:** YARN
- **Dataset:** NCDC Historical Weather Dataset

Conclusion:

The NCDC weather dataset was successfully processed using the Hadoop MapReduce framework to identify the maximum temperature recorded for each year. The Mapper extracted the year and temperature information from individual weather records and generated intermediate (Year, Temperature) key-value pairs. Hadoop's Shuffle and Sort phase grouped all temperature values

belonging to the same year, after which the Reducer determined the maximum temperature for each year.

The analysis showed an increasing trend in the yearly maximum temperatures in the given dataset, **with 2022 recording the highest temperature of 45°C**. This practical demonstrated how Hadoop and MapReduce can efficiently process large-scale climate datasets through distributed storage, parallel processing, scalability, data locality, and fault tolerance.

Compared with traditional sequential processing, MapReduce provides a more scalable and efficient approach for analyzing massive weather datasets. The same processing approach can also be extended to calculate minimum temperatures, average yearly temperatures, rainfall trends, and other long-term climate patterns.

Github Repo:

https://github.com/Gitanshu1504/24DIT070_BDA_Practicals_2026_SHP